

[Atomic] Split Order Feature

1. Context

We allow the flexibility of cutting down a large job into multiple splits, which would be useful when single vehicle does not have enough capacity to handle those large load jobs in one time.

2. API Changes

Changes

Split order only apply to single jobs, from the input we allow the following three fields

Job Object		
Field	Type	Remark
split	Split Object	The split object as displayed below
id		
...		

Split Object		
Field	Type	Remark
can_be_split	bool	default false
max_splits	int	maximum number of splits
quant	float	min quantity of each split in terms of job load ratio, applies only when the problem load dimension is 1 (multi dimension case definition not clear enough).

In the response, we need an indicator for split jobs. In particular the parameter `split_indicator` takes this role. In additional, we need extra term `pickup` and `delivery` term to indicate the actual load the split order take.

Important note: the sub-job share the same job id as the original one.

Route Step Object		
Field	Type	Remark
split_indicator	int	should be less than the original max_splits value of the job, starting from 0
pickup	int array	sub volume the job carry, the sum should be equal to original value

delivery	int array	sub volume the job carry, the sum should be equal to original value
----------	-----------	---

Let's take a simple case as an example, a delivery job (id=1) has delivery volume [2] and we only have two vehicles (id=1, id=2) with capacity=[1] for both. If the job is not splittable, then the only choice is to mark it as unassigned

```
{
  "routes": [
    {
      "vehicle": 1,
      "steps": [
        {
          "type": "start",
          "location": [
            77.0828432250535,
            28.739178805785464
          ],
          "location_index": 0,
          "load": [
            1
          ]
        },
        {
          "type": "job",
          "location": [
            77.10119068307893,
            28.725320281664338
          ],
          "location_index": 1,
          "id": 1,
          "load": [
            0
          ],
          "delivery": [
            1
          ],
          "split_indicator": 0
        },
        {
          "type": "end",
          "location": [
            77.0828432250535,
            28.739178805785464
          ],
          "location_index": 0,
          "load": [
            0
          ]
        }
      ],
      "delivery": [
        1
      ]
    },
  ],
}
```

```

{
  "vehicle": 2,
  "steps": [
    {
      "type": "start",
      "location": [
        77.0828432250535,
        28.739178805785464
      ],
      "location_index": 0,
      "load": [
        1
      ]
    },
    {
      "type": "job",
      "location": [
        77.10119068307893,
        28.725320281664338
      ],
      "location_index": 1,
      "id": 1,
      "load": [
        0
      ],
      "delivery": [
        1
      ],
      "split_indicator": 1
    },
    {
      "type": "end",
      "location": [
        77.0828432250535,
        28.739178805785464
      ],
      "location_index": 0,
      "load": [
        0
      ]
    }
  ],
  "delivery": [
    1
  ]
}

```

Limitations

Limit to jobs only (shipments not able to be supported)

3. Tech Design

Executor

We consider splittable jobs allocation into two steps

1. First allocate all jobs with not-splittable jobs as many as possible and find a solution with minimum costs
2. For remaining unassigned jobs, if they are splittable, we try to split then into multiple dummy jobs and try allocating again.

4. Testing Suggestions

Test cases can follow the following guideline

1. Design particular input format which does not follow our required rule, to check whether expected error messages are returned
2. Design a case with job delivery volume is high value, e.g., 5 times of vehicle capacity, when the job `can_be_split=false` or not specified, check whether the job is unassigned. When `can_be_split=true` and we have 5 vehicles, check whether we return proper solution route.

5. Notes

N.A